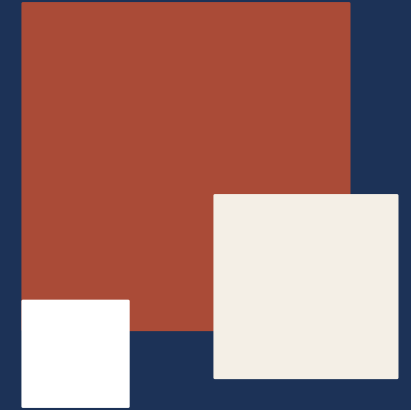


MINERÍA DE DATOS · CICLO 2026A

Data Warehousing y Cubos OLAP

Almacenes de datos, modelo multidimensional y procesamiento analítico en línea



Universidad Politécnica de Chiapas
Ingeniería de Software · 9.º semestre

Lectura base
Han, Pei & Tong (2023) — Data Mining, 4.^a ed., Cap. 3

Agenda de la clase

01

Fundamentos

Por qué un almacén; OLTP vs OLAP y tipos de repositorio.

02

Arquitectura

Tres capas, ETL, data marts y data lakes.

03

Modelo multidimensional

El cubo de datos, cuboides, esquemas y jerarquías.

04

Operaciones OLAP

Roll-up, drill-down, slice & dice y pivot.

05

Implementación física

Índices, almacenamiento columnar y materialización.

06

Práctica

Producción, snippets de código SQL y mini-proyecto en DuckDB.

¿Por qué un Data Warehouse?

Las bases de datos operacionales y el análisis de negocio compiten por los mismos recursos. Separarlos es la decisión de diseño que da origen al almacén de datos.



Sistema operacional (OLTP)

- Registra el día a día: una compra, un alta de cliente, un envío.
- Optimizado para insertar, borrar y modificar pocos registros.
- Miles de transacciones concurrentes; datos actuales y detallados.



Análisis de negocio (OLAP)

- Vistas históricas, actuales y predictivas del negocio.
- Consultas que unen muchas tablas y agregan en grandes grupos.
- Ejecutadas sobre el sistema operacional, lo degradan gravemente.

Analogía La caja registradora atiende clientes en segundos (operación). El gerente que estudia las ventas del trimestre necesita otra mesa: si analiza sobre la caja, la fila se detiene.

OLTP frente a OLAP

Criterio	OLTP — Procesamiento transaccional	OLAP — Procesamiento analítico
Usuarios	Operativos: cajeros, clientes, sistemas	Knowledge workers: analistas y directivos
Función	Ejecutar las operaciones del negocio	Obtener insights y apoyar decisiones
Datos	Actuales, detallados, atómicos	Históricos, resumidos, multi-granularidad
Diseño de BD	Entidad-relación, orientado a la aplicación	Estrella o copo de nieve, orientado al sujeto
Accesos	Transacciones cortas; requiere control de concurrencia	Consultas complejas, mayormente de solo lectura
Vista	Datos de un departamento, sin historia	Integra versiones y múltiples organizaciones

La diferencia de estructura, contenido y uso es la razón por la que ambos sistemas se mantienen físicamente separados.

Database, Data Warehouse y Data Lake

Son tres repositorios distintos. Cada uno resuelve una etapa diferente del ciclo de vida de los datos.



Base de datos

OPERAR

QUÉ ES

Sistema que almacena los datos vivos del negocio. Estructurado y normalizado; el esquema se define al escribir.

SIRVE PARA

Ejecutar las operaciones del día a día —ventas, altas, pagos— con transacciones rápidas y consistentes.

Ejemplos PostgreSQL · MySQL · Oracle

“¿Qué está pasando ahora?”



Data Warehouse

ANALIZAR

QUÉ ES

Repositorio que integra datos históricos de muchas fuentes, ya limpios y estructurados para el análisis.

SIRVE PARA

Responder preguntas de negocio y apoyar decisiones, sin afectar a los sistemas operativos.

Ejemplos Snowflake · BigQuery · Redshift

“¿Qué pasó y por qué?”



Data Lake

EXPLORAR

QUÉ ES

Almacén masivo que guarda todos los datos en su formato natural, sin transformarlos al ingresar.

SIRVE PARA

Conservar todo —texto, imágenes, logs, JSON— para ciencia de datos y machine learning futuros.

Ejemplos Amazon S3 · Azure Data Lake · HDFS

“¿Qué podríamos descubrir?”

En conjunto la base de datos opera, el data warehouse analiza y el data lake explora. En una organización moderna los tres conviven y se alimentan entre sí.

Definición: las cuatro claves de Inmon

“Un data warehouse es una colección de datos orientada al sujeto, integrada, variante en el tiempo y no volátil, en apoyo a la toma de decisiones.”



Orientada al sujeto

Organizada en torno a sujetos clave —cliente, producto, ventas— y no a operaciones cotidianas.



Integrada

Construida a partir de fuentes heterogéneas, con limpieza y convenciones de nombres unificadas.



Variante en el tiempo

Cada estructura lleva un elemento temporal; abarca un historial amplio (p. ej. 5–10 años).



No volátil

Almacén separado de los datos operacionales: solo carga y consulta; los datos no se borran.

En síntesis Un repositorio semánticamente consistente y persistente: la implementación física de un modelo de datos para soporte a decisiones.

Arquitectura de tres capas



Capa superior — Cliente front-end

Herramientas de consulta, reportes, visualización, análisis y minería de datos (tendencias, predicción).



Capa media — Servidor OLAP

Presenta datos multidimensionales. Implementado como ROLAP, MOLAP o HOLAP.



Capa inferior — Servidor del almacén

Base de datos relacional o key-value. Aloja el repositorio de metadatos y recibe los datos vía ETL.

PIEZAS TRANSVERSALES

Repositorio de metadatos

“Datos sobre los datos”: esquema, dimensiones, linaje, reglas de resumen y mapeos.

Back-end / ETL

Extracción, limpieza, transformación, carga y refresco del contenido.

Herramientas de gestión

Monitoreo, gobernanza y administración del almacén.

ETL: poblar y refrescar el almacén

Los datos extraídos de las fuentes rara vez sirven tal cual. El proceso ETL los prepara antes de cargarlos al almacén.



Refresco (Refresh) cuarta tarea continua del ETL: propaga las actualizaciones de las fuentes hacia el almacén de forma periódica.

Almacén corporativo y data marts



Enterprise Warehouse

- Reúne toda la información de la organización, de alcance multifuncional.
- Integra datos de uno o varios sistemas operacionales y proveedores externos.
- Tamaño de cientos de GB hasta terabytes; modelado a nivel empresa.
- Su construcción puede tomar años.

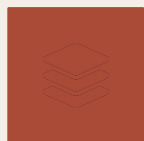


Data Mart

- Subconjunto de datos de valor para un grupo específico, p. ej. un departamento.
- Alcance acotado a sujetos seleccionados; datos típicamente resumidos.
- Independiente (de fuentes propias) o dependiente (del almacén corporativo).
- Se implementa en semanas, no en meses.

Almacén virtual tercera opción: un conjunto de vistas sobre las bases operacionales. Fácil de construir, pero carga de trabajo extra sobre los servidores operacionales.

Data Lakes



Repositorio único de todos los datos de la empresa en su formato natural

Relacionales, semiestructurados (JSON, CSV), no estructurados (PDF, correo) y binarios (imagen, audio, video).

Data Warehouse

Esquema al escribir · top-down · estructurado y centralizado

Data Lake

Esquema al leer · bottom-up · prototipado rápido y democrático

En la práctica empresarial se combinan ambos para aprovechar lo mejor de cada enfoque.

CAPAS DE ALMACENAMIENTO

Sandbox

Experimentos de científicos de datos

opcional

Aplicación

Lógica de negocio; capa de producción

Depurada (cleansed)

Datos limpios; accesible a usuarios

Estandarizada

Formatos óptimos para limpieza

opcional

Datos crudos (raw)

Ingestión en formato nativo

OLAP: qué significa y por qué un «cubo»

OLAP es la categoría de herramientas para el análisis interactivo de datos. Organiza la información en una estructura llamada cubo.



■ ¿POR QUÉ SE LE LLAMA «CUBO»?

Cada dimensión del análisis —tiempo, producto, ubicación— es un eje. Los datos se ordenan a lo largo de todos los ejes a la vez; con tres ejes, las celdas llenan un volumen: un cubo.



Con 4 o más dimensiones el cubo se vuelve un hipercubo; el nombre «cubo» se conserva por intuitivo.

El cubo de datos

El núcleo del análisis multidimensional es el cómputo eficiente de agregaciones sobre muchos conjuntos de dimensiones.



Dimensiones

Las perspectivas del análisis: tiempo, ítem, sucursal, ubicación. Cada una tiene su tabla de dimensión.



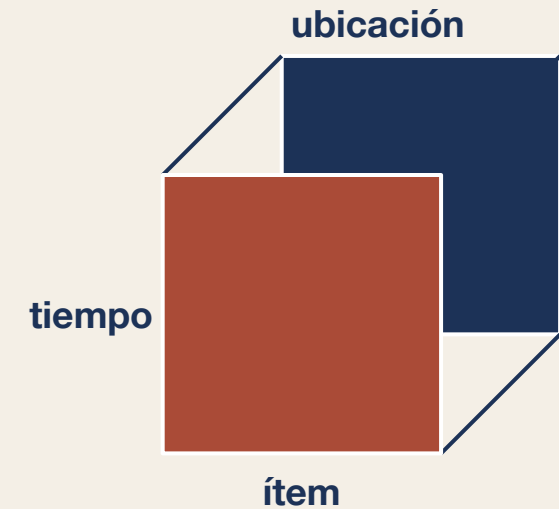
Hechos / medidas

Las mediciones del sujeto: dollars_sold, units_sold. Numéricas y guardadas en la tabla de hechos.



Celda

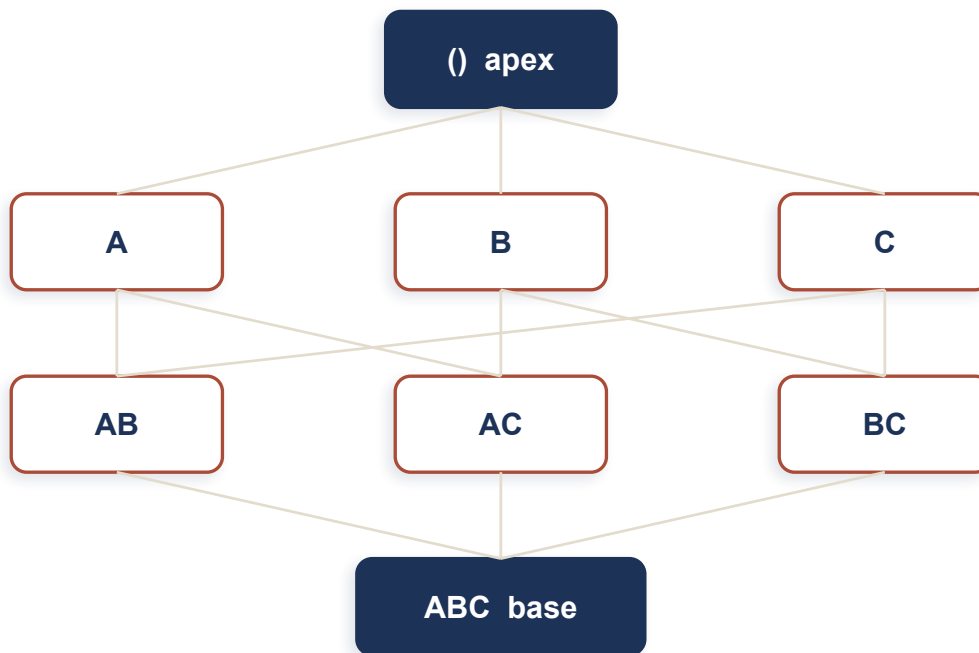
Un punto del espacio del cubo, definido por pares dimensión–valor; contiene el valor agregado.



Una tabla es un cubo 2-D. Un cubo es n -dimensional: una serie de cubos de $(n-1)$ dimensiones.

Cuboides y el lattice del cubo

Cada group-by posible es un **cuboid**. El conjunto de todos los cuboides forma un lattice: eso es el cubo de datos.



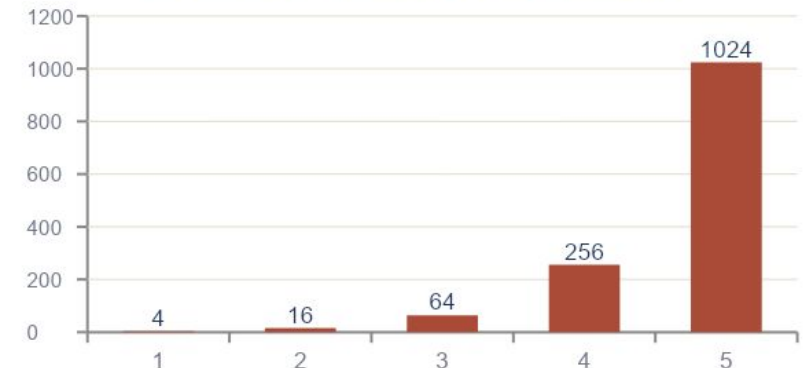
¿CUÁNTOS CUBOIDES?

Sin jerarquías: 2^n cuboides

Con jerarquías: $\prod (L_i + 1)$

base = mínima agregación · apex = máxima agregación (todo)

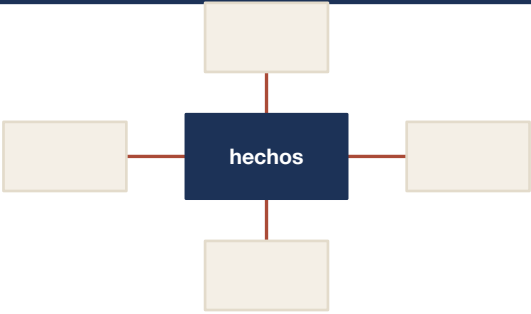
Crecimiento exponencial del nº de cuboides



Esquemas: estrella, copo de nieve, constelación



Esquema estrella

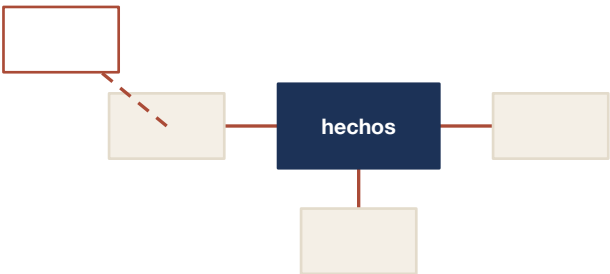


Una tabla de hechos central y una tabla por dimensión, sin normalizar. El grafo recuerda una estrella.

El más popular: simple y rápido de explorar.



Copo de nieve

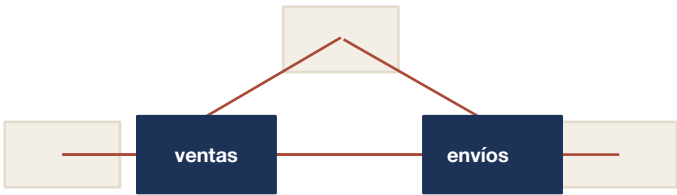


Variante donde algunas dimensiones se normalizan en tablas adicionales.

Reduce redundancia, pero más joins frenan el browsing.



Constelación de hechos



Varias tablas de hechos comparten tablas de dimensión (galaxy schema).

Para aplicaciones sofisticadas con múltiples sujetos.

Jerarquías de concepto

Una jerarquía de concepto define una secuencia de mapeos de conceptos de bajo nivel a conceptos más generales: permite analizar los datos en distintos niveles de abstracción.

Orden total — dimensión location



$\text{street} < \text{city} < \text{province_or_state} < \text{country}$

Muchas jerarquías están implícitas en el esquema de la base de datos.

Orden parcial — dimensión time



$\text{day} < \{\text{month} < \text{quarter}; \text{week}\} < \text{year}$

También por agrupamiento discretizar valores —p. ej. rangos de price— genera jerarquías set-grouping definidas por el usuario o el experto.

Medidas: tres categorías

Una medida es una función numérica evaluada en cada celda del cubo. Se clasifica según el tipo de función de agregación que la calcula.



Distributiva

Se computa en forma distribuida: agregar las particiones equivale a agregar todo el conjunto.

`sum( )` `count( )` `min( )` `max( )`

Cómputo eficiente y particionable.



Algebraica

Función algebraica de M argumentos, cada uno obtenido con una función distributiva.

`avg( ) = sum( ) / count( )` · `stddev( )`

Aún eficiente; combina distributivas.



Holística

No existe cota constante de almacenamiento para describir el sub-agregado.

`median( )` `mode( )` `rank( )`

Difícil de computar; suele aproximarse.

La mayoría de los cubos a gran escala usan medidas distributivas y algebraicas por su cómputo escalable.

Operaciones OLAP

Un cubo permite ver los datos desde múltiples perspectivas. Estas operaciones hacen interactiva la consulta y el análisis.



Roll-up (drill-up)

Agrega: sube por la jerarquía de una dimensión o reduce dimensiones.

EJEMPLO

ciudad → país



Drill-down

Inverso del roll-up: navega de datos generales a más detallados.

EJEMPLO

trimestre →
mes



Slice & dice

Slice: selección en una dimensión. Dice: selección en dos o más.

EJEMPLO

time = «Q1»



Pivot (rotate)

Rota los ejes de la vista para presentar los datos de otra forma.

EJEMPLO

ítem ↔
ubicación

Otras operaciones: drill-across (varias tablas de hechos) y drill-through (hasta las tablas relacionales de base).

Indexación OLAP: bitmap y join index



Índice bitmap

- Un vector de bits por cada valor del dominio del atributo.
- Ideal para dominios de baja cardinalidad: comparación, join y agregación se reducen a aritmética de bits.
- Se extiende a bit-sliced indexing para datos numéricos.



Join index

- Precalcula y guarda los pares de identificadores del resultado del join.
- Evita unir una y otra vez la tabla de hechos con las de dimensión.
- Reduce drásticamente el costo de I/O en consultas de agregación.

≈ 800×

menos I/O al contar con
bitmap frente a escanear la
tabla completa

≈ 90 %

de I/O ahorrado al usar join
index para localizar las tuplas
relevantes

Almacenamiento columnar



Almacenar columna por columna

- Una consulta OLAP suele agregar muchos registros usando pocos atributos.
- Si los datos se guardan fila por fila, hay que escanear todo el registro para usar una fracción mínima.
- La base columnar guarda cada atributo en bloques contiguos: solo lee las columnas necesarias.
- Usa bitmaps para encadenar resultados intermedios entre columnas.

Contrapartida separar transacciones en columnas y comprimir las la hace costosa para cargas OLTP.

EJEMPLO DEL CAPÍTULO

Ingreso familiar promedio de clientes por provincia
— tabla de 10 millones de registros.

1 GB

Almacenamiento por filas

Escanea la tabla entera; solo el 3 % es útil.

40 MB

Almacenamiento por columnas

Lee tres columnas: gender, province e income.

Materialización del cubo de datos

Sin materialización

Nada se precalcula: agregados multidimensionales al vuelo, extremadamente lentos.

Materialización total

Se precalculan todos los cuboides (full cube): requiere muchísimo espacio.

Materialización parcial

Se calcula un subconjunto de cuboides o celdas: equilibrio espacio–tiempo.



Iceberg cube

Materializa solo las celdas cuyo agregado supera un umbral mínimo (min_sup). Las celdas triviales —la mayoría— se descartan.

```
compute cube sales_iceberg as
  select month, city, customer_group, count(*)
  from salesInfo cube by month, city, customer_group
  having count(*) >= min_sup
```

ALGORITMOS DE CÁLCULO

MultiWay cubo completo (MOLAP) con arreglos y agregación simultánea.

BUC iceberg cubes de arriba hacia abajo; poda por antimonotonía.

Shell-fragment fragmenta el cubo para OLAP de alta dimensionalidad.

Data Warehousing y OLAP en producción

Los conceptos del capítulo siguen vigentes: así se ven hoy en la industria y en el stack del curso.



DuckDB

Motor OLAP columnar embebido: ejecuta consultas analíticas sobre archivos sin servidor. Es el motor del mini-proyecto de esta unidad.



Cloud data warehouses

Snowflake, BigQuery y Redshift son almacenes columnares que separan el cómputo del almacenamiento para escalar.



El esquema estrella perdura

Sigue siendo el estándar de modelado dimensional; herramientas como dbt versionan las transformaciones del ETL.



Data lakehouse

Converge data warehouse y data lake: tablas estructuradas y gobernadas directamente sobre el almacenamiento del lake.

Definir el esquema estrella en SQL

Modelar el almacén empieza por declarar la tabla de hechos y sus dimensiones.

-- DIMENSIONES — atributos descriptivos y jerarquías

```
CREATE TABLE dim_time (  
  time_key INTEGER PRIMARY KEY,  
  day DATE, month VARCHAR, quarter VARCHAR, year INTEGER );
```

```
CREATE TABLE dim_item (  
  item_key INTEGER PRIMARY KEY,  
  item_name VARCHAR, brand VARCHAR, category VARCHAR );
```

```
CREATE TABLE dim_location (  
  loc_key INTEGER PRIMARY KEY,  
  city VARCHAR, state VARCHAR, country VARCHAR );
```

-- HECHOS — claves foraneas + medidas numericas

```
CREATE TABLE fact_sales (  
  time_key INTEGER REFERENCES dim_time(time_key),  
  item_key INTEGER REFERENCES dim_item(item_key),  
  loc_key INTEGER REFERENCES dim_location(loc_key),  
  dollars_sold DECIMAL(12,2),  
  units_sold INTEGER );
```

SQL · DuckDB

CÓMO LEERLO

Tabla de hechos

Contiene solo claves foráneas y medidas numéricas aditivas.

Dimensiones

Atributos descriptivos; alojan las jerarquías de concepto.

Las FK son los rayos

Cada REFERENCES une un rayo de la estrella con el centro.

Construir el cubo: CUBE, ROLLUP y GROUPING SETS

Una sola consulta agregada genera los cuboides que materializan el cubo.

```
-- CUBE: materializa los 2^n cuboides de agregacion
SELECT t.year, l.country, i.category,
       SUM(f.dollars_sold) AS total
FROM   fact_sales f
JOIN   dim_time t   ON f.time_key = t.time_key
JOIN   dim_location l ON f.loc_key = l.loc_key
JOIN   dim_item i   ON f.item_key = i.item_key
GROUP BY CUBE (t.year, l.country, i.category);

-- ROLLUP: subtotales jerarquicos, no todas las combinaciones
SELECT country, state, city, SUM(dollars_sold)
FROM   v_sales
GROUP BY ROLLUP (country, state, city);

-- GROUPING SETS: materializacion parcial a la medida
SELECT year, country, SUM(dollars_sold)
FROM   v_sales
GROUP BY GROUPING SETS ((year), (country), (year,country), ());
```

CÓMO LEERLO

CUBE

Genera todos los cuboides, del apex a la base, en una pasada.

ROLLUP

Solo los subtotales de la jerarquía: país, estado, ciudad.

GROUPING SETS

Defines exactamente qué cuboides quieres materializar.

Operaciones OLAP en DuckDB

Roll-up, drill-down, slice & dice y pivot, en código ejecutable.

```
-- Vista que une hechos y dimensiones (el "cubo logico")
CREATE VIEW v_sales AS
SELECT t.year, t.month, i.category, l.country, f.dollars_sold
FROM fact_sales f
JOIN dim_time t USING (time_key)
JOIN dim_item i USING (item_key)
JOIN dim_location l USING (loc_key);

-- ROLL-UP / DRILL-DOWN: cambiar el nivel de la jerarquia
SELECT country, SUM(dollars_sold) FROM v_sales GROUP BY country;
SELECT year, month, SUM(dollars_sold) FROM v_sales
GROUP BY year, month;

-- SLICE & DICE: filtrar una o varias dimensiones
SELECT category, SUM(dollars_sold) FROM v_sales
WHERE year = 2025 GROUP BY category;

-- PIVOT: rotar ejes (sintaxis nativa de DuckDB)
PIVOT v_sales ON country USING SUM(dollars_sold) GROUP BY category;
```

CÓMO LEERLO

La vista v_sales

Une hechos y dimensiones: el cubo sobre el que se navega.

Roll-up / drill-down

Sube o baja por la jerarquía agregando o detallando.

Slice & dice

WHERE sobre una (slice) o varias (dice) dimensiones.

PIVOT

DuckDB rota los ejes sin escribir el cruce a mano.

Buenas prácticas y anti-patrones



Buenas prácticas

- Separar físicamente los sistemas OLTP y OLAP.
- Definir la granularidad de la tabla de hechos desde el diseño.
- ETL con limpieza, validación y reglas de negocio explícitas.
- Usar el esquema estrella como punto de partida del modelado.
- Materializar de forma selectiva: iceberg cubes y shell fragments.
- Gobernar el data lake: catálogo, linaje y seguridad por capa.



Anti-patrones

- Ejecutar consultas OLAP sobre la base de datos operacional.
- Normalizar en exceso: el snowflake añade joins que frenan la exploración.
- Materializar el cubo completo en alta dimensionalidad.
- Jerarquías de concepto mal definidas o incoherentes.
- Cargar un data lake sin gobernanza: se convierte en data swamp.
- Confiar en los datos sin verificar su calidad ni su linaje.

Ejercicio: mini data cube en DuckDB



Consigna

- Construir una tabla de hechos sales(time, item, location, dollars_sold).
- Cargar datos con DuckDB y modelar un cubo de 3 dimensiones.
- Resolver roll-up, drill-down, slice y dice con GROUPING SETS / CUBE.
- Comparar una medida distributiva (sum) y una holística (median).

```
SELECT location, item, SUM(dollars_sold)
FROM sales GROUP BY CUBE(location, item);
```

ENTREGA Y DEFENSA

Repositorio del curso

github.com/RamsesCamas/DataMining_Slices_and_Code

Entregar notebook + script SQL reproducible.

Defensa oral

Explicar el código en 3 minutos.

■ SÍNTESIS Y PRÓXIMA CLASE

El almacén separa operación de análisis; el cubo de datos y las operaciones OLAP lo hacen interactivo; **índices, columnas y materialización lo hacen rápido.**

SIGUE

Clasificación y Regresión

Examen: 26 de mayo de 2026